告别“伪智能”代码：用 Spec + RAG 打造真正懂你的AI程序员

导购&商详团队 大淘宝技术 2026年4月8日 01:06 浙江 15人



本文提出了一种结合“规范（Spec）”与“检索增强生成（RAG）”的全新AI编程范式，旨在解决当前AI编程中常见的如幻觉、上下文缺失和逻辑不连贯等问题。文章指出，单纯依赖大模型的自然语言理解往往导致代码生成不准确，而通过引入结构化的开发规范（Spec）作为明确指令，并配合RAG技术实时检索项目特有的代码库、文档和最佳实践，可以赋予AI真正的“项目感知力”。这种模式让AI从通用的代码生成器转变为懂业务、懂架构的专属程序员，显著提升了代码生成的准确性、可维护性及与现有系统的融合度，为构建高质量、低幻觉的AI辅助开发流程提供了切实可行的落地方案。

01 引言：AI Coding 提升代码质量的关键——知识库的深度建设

在当前 AI Coding 快速普及的背景下，业界普遍面临一个核心矛盾：模型“能写”≠“写得对”。尤其在高频迭代、强业务耦合的场景中，代码的正确性、可维护性和一致性远比“能生成”更重要。

要突破这一瓶颈，关键在于让 AI 不仅“会写”，更要“懂上下文”——即深刻理解特定项目的技术契约、业务语义与工程惯例。为此，我们提出构建一套高质量、结构化的知识体系，作为 AI Coding 的“认知基础设施”。这一体系包含两个互补维度：

- **Spec 知识库**：基于规范驱动开发（Spec-Driven Development, SDD）沉淀的项目级契约与规则；
- **RAG（Retrieval-Augmented Generation）知识库**：动态接入外部文档、历史方案、最佳实践等非结构化或半结构化知识。

二者共同构成 AI 的“上下文感知能力”，使其不仅能理解“要做什么”，更能精准把握“怎么做才对”。Spec 提供强约束的“硬规则”，RAG 提供灵活丰富的“软上下文”。本文将系统阐述这一知识体系的构想、落地现状与未来演进。

Spec简介：AI Coding 的“宪法”

- 什么是Spec?

Spec（Specification，规范） 是对软件系统行为、接口、数据格式或业务规则的精确、无歧义、可验证的描述。在 AI Coding 中，SPEC 扮演着“宪法”的角色——它明确告诉 AI：“代码必须满足哪些条件才算正确”，而不是依赖模型自行猜测、模仿或凭“感觉”生成。

其核心价值体现在俩个层面：

- **规范即契约：**SPEC 是开发者、AI Agent 与系统之间达成的共识性“契约”，清晰界定“做什么”（What）、“为什么做”（Why）以及“如何做”（How）。
- **AI 的指令集：**为大语言模型（LLM）提供明确、结构化的上下文，显著减少幻觉（hallucination），提升生成代码的准确性与一致性。

- Spec Coding VS Vibe Coding

随着AI编程普及，开发者分化出两种范式：Vibe Coding 依赖直觉和意图模仿，快但不可靠；Spec Coding 严格遵循规范，可靠且易维护，适合企业级和高要求场景。选择关键在于权衡速度与确定性。对比如下：

维度	Vibe Coding	Spec Coding
依据	依赖开发者或 LLM 对“大致感觉”的主观理解	严格遵循书面化、结构化的规范文档
生成逻辑	“我觉得应该这样写”——基于统计模式与风格模仿	“规则要求必须这样写”——基于契约约束与条件满足
可靠性	易产生幻觉、遗漏边界条件，结果不可预测	行为可预期、可验证、可审计
维护成本	初期快，后期高：需求变更或缺陷修复需反复试错	初期需投入，后期低：只需更新 SPEC，AI 可自动适配新规则

适用场景	快速原型、个人项目、探索性编码	企业级开发、安全关键系统、合规性要求高的场景

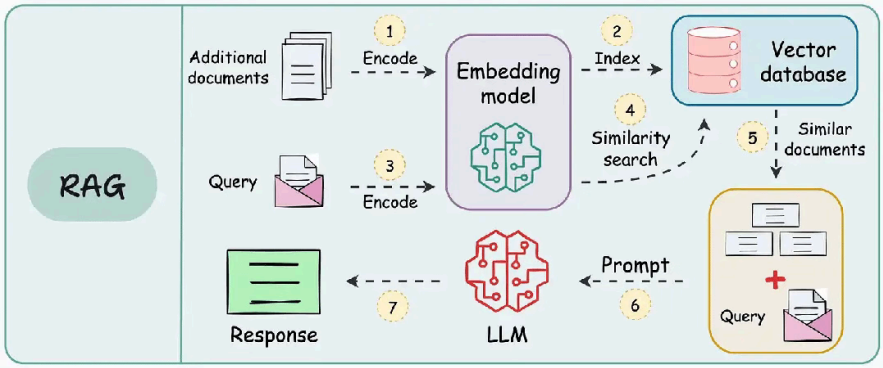
RAG简介：让 LLM “看得见”你的私有知识

- 什么是RAG?

RAG（Retrieval-Augmented Generation） 是一种结合“搜索”与“生成”的技术，使 LLM 在回答问题时能参考外部知识库，生成更准确、可溯源、事实性强的答案。

其工作流程分为三步：

- **Retrieve（检索）**：根据用户查询，从外部知识库（如文档、数据库）中检索最相关的文本片段；
- **Augment（增强）**：将检索结果拼接到原始 Prompt 中，形成增强上下文；
- **Generate（生成）**：LLM 基于增强后的上下文生成最终回答。



- 为什么需要 RAG?

尽管 LLM 能力强大，但在实际应用中仍面临诸多局限：知识静态，无法访问私有或实时数据；容易产生幻觉，输出内容可靠性存疑；更新成本高昂，需要重新训练或微调模型。

RAG 提供了一种创新性的解决思路。其核心理念是：不求模型“记住一切”，而让它“懂得查找”。通过按需检索可信知识源，RAG 显著提升了生成内容的准确性、时效性与可溯源性。

下表对比了 LLM 与 RAG 在核心维度上的差异，直观展现 RAG 如何有效弥补 LLM 的实际应用短板。

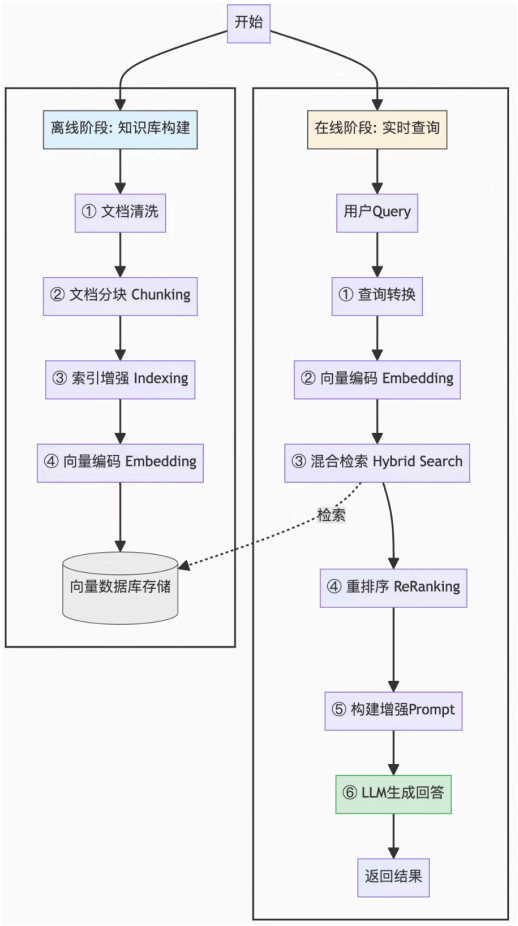
能力维度	传统 LLM	RAG
知识范围	仅限训练数据截止前的公开知识，无法访问私有或实时数据	可结合外部知识库，支持私有文档、最新规范、实时信息
知识更新	需微调或重新训练，成本高、周期长	仅需更新检索库，无需改动模型，低成本快速迭代
幻觉风险	高：无答案时易编造虚假信息	低：基于真实检索结果生成，答案有据可依
垂直领域表现	泛化能力强，但专业深度不足	通过领域知识库显著提升专业性与准确性
回答相关性	依赖 prompt 完整性，易遗漏关键上下文	动态注入高相关检索片段，增强上下文匹配度
安全与合规	私有数据需输入上下文，存在泄露风险	原始数据保留在检索层，可通过权限控制保障数据安全
部署与维护成本	模型固定，但扩展知识需重训	轻量灵活，仅维护检索索引，适合持续演进

- RAG工作流程

RAG 系统包含两个核心阶段：离线阶段构建知识库，在线阶段处理实时查询。以下详细介绍高级 RAG 系统在这两个阶段的关键实现流程。

- **离线阶段：知识库构建**
  - 文档预处理：对原始文档进行清洗，去除噪声和无关信息
  - 智能分块：将长文档切分成语义完整、大小适中的文本片段
  - 索引增强：为文本片段添加元数据（如来源、时间、分类）和关键词标签
  - 向量化编码：使用嵌入模型将文本转换为向量表示
  - 数据入库：将向量与原文一同存储到向量数据库中，构建完整的检索索引
- **在线阶段：实时查询**
  - 查询优化：对用户问题进行改写、扩展和消歧，提升查询表达的准确性
  - 向量化检索：将优化后的查询转换为向量表示
  - 混合召回：在向量数据库中执行语义检索与关键词匹配的混合检索策略，召回候选文档集

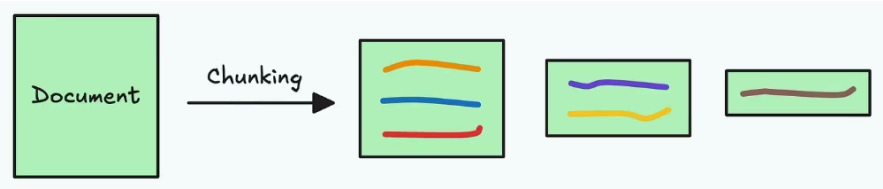
- 重排序：使用精排模型对候选文档进行相关性打分与重新排序
- 结果返回：将最相关的文档片段提供给大模型用于答案生成



- RAG关键技术

分块（Chunking）

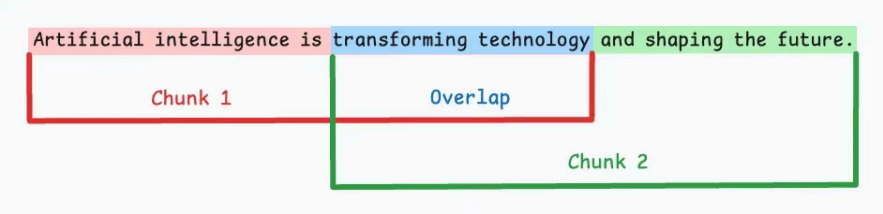
分块是连接原始知识库与 LLM 有限上下文窗口的关键环节。其核心目标是将大型文档切分为语义完整、长度适中的文本块，既满足嵌入模型对输入长度的限制，又确保每个片段保留足够的上下文信息以支持精准检索。这一过程不仅直接影响嵌入质量和向量检索的准确性，还从根本上决定了最终生成回答的相关性与可靠性。



以下是五种用于RAG的分块策略：

1. 固定大小分块（Fixed-size chunking）

固定大小分块是一种将文本按预设长度（如字符、单词或 token 数量）均匀切分为等长片段的策略。



优点：

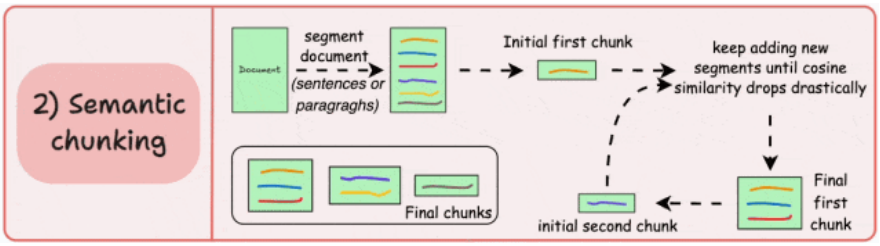
- 实现简单，易于自动化；
- 所有块大小一致，便于批量处理和向量化；
- 可通过设置重叠区域（overlap）部分缓解上下文断裂问题。

缺点：

- 容易在句子或语义单元中间切断，破坏语义完整性；
- 关键信息可能被拆分到多个块中，降低检索相关性和生成质量；

2. 语义分块（Semantic Chunking）

语义分块是一种基于文本内在语义结构的智能分块方法。它首先将文档按自然语言单元（如句子或段落）切分，然后逐段计算嵌入向量，并通过余弦相似度衡量相邻段落的语义关联性。若相似度高，则合并为一个块；一旦相似度显著下降，即视为语义边界，开启新块。



优点：

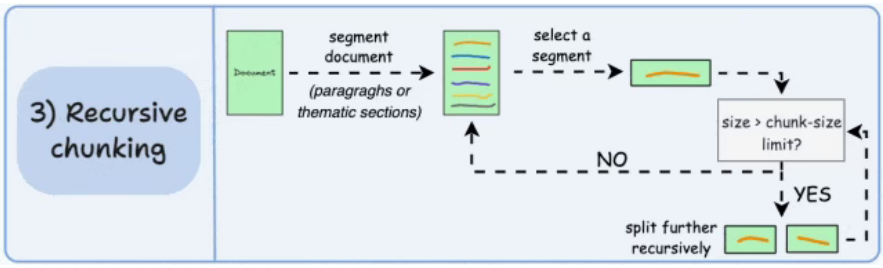
- 保留完整的语义单元和语言逻辑，避免信息割裂；
- 每个块包含更连贯、更丰富的上下文信息，显著提升检索准确性和生成质量；

缺点：

- 依赖相似度阈值来判断语义边界，而该阈值难以统一，需针对不同文档调整；

3. 递归分块（Recursive Chunking）

递归分块是一种层次化、自上而下的分块策略。它首先利用文本中天然的分隔符（如句号、换行、标题等）进行粗粒度切分，然后对超出预设长度限制的块递归地进一步拆分，直到所有子块都满足大小要求；若某一块已符合长度限制，则保留原样，不再切割。



优点：

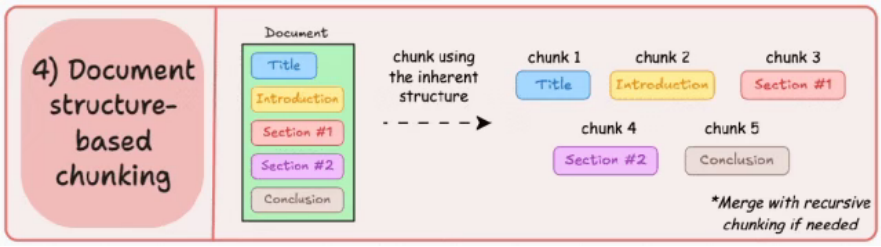
- 相比固定大小分块，能更好地保留完整句子和上下文逻辑，助于保持语义完整性；

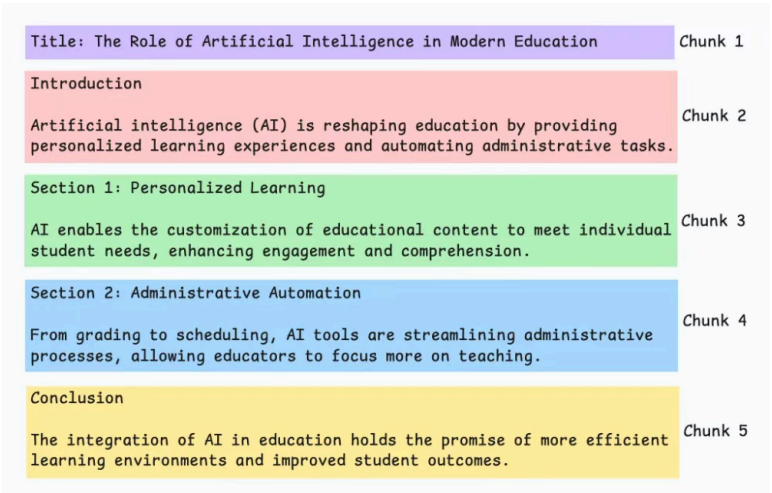
缺点：

- 计算开销略高，尤其在深层递归时；

4. 基于文档结构分块（Document structure-based chunking）

基于文档结构的分块（Document Structure-based Chunking） 是一种利用文档自身层级结构（如标题、章节、段落等）来划定分块边界的策略。它将每个逻辑单元（例如一个章节或带标题的小节）作为一个独立的文本块，从而在最大程度上保留原文的组织结构和语义完整性。





优点：

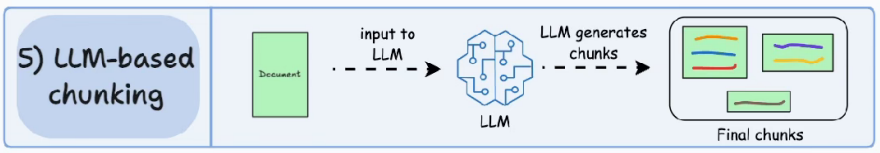
- 紧密贴合文档的原始逻辑结构，块内信息高度相关；
- 对结构化文档（如技术文档、论文、说明书）效果尤佳。

缺点：

- 依赖文档具备清晰、规范的结构，对格式混乱或无结构的文本（如纯文本日志、网页抓取内容）效果有限；
- 生成的块长度可能差异很大，部分块可能超出嵌入模型或LLM的token限制；

5. 基于LLM分块（LLM-based Chunking）

基于大语言模型的分块（LLM-based Chunking） 是一种利用大语言模型（LLM）自身理解能力来生成语义连贯、独立文本块的方法。通过精心设计的提示（prompt），可以引导 LLM 将原始文档重构成多个语义完整、边界清晰的片段。



优点：

- LLM 能深入理解上下文和逻辑关系，生成的文本块语义完整、边界合理

缺点：

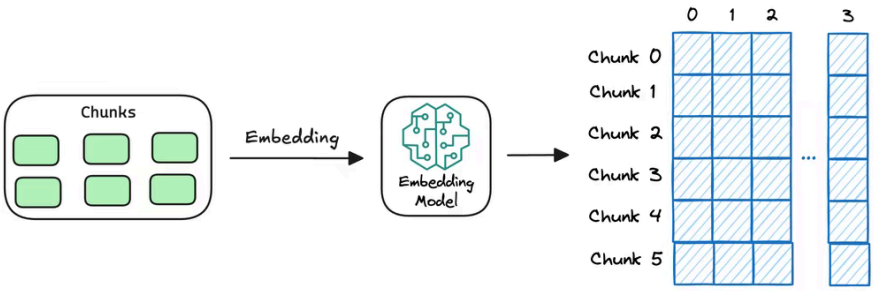
- 计算成本最高，需对全文调用 LLM，推理开销大；
- 受限于 LLM 自身的上下文窗口长度，处理超长文档时需额外分段策略；

- 向量编码（Embedding）

Embedding 是 RAG系统实现语义级检索的核心技术，主要应用于检索阶段。其核心思想是将人类可读的非结构化信息（如自然语言文本）转化为机器可理解、可计算的高维向量表示，从而在向量空间中高效、精准地捕捉和匹配语义信息。

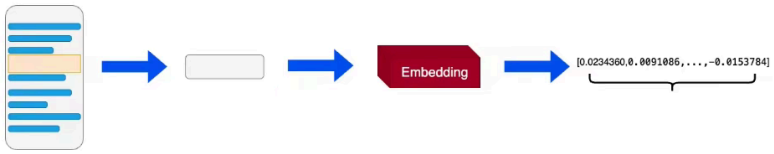
其关键作用体现在以下两个方面：

- **向量空间映射：**将用户查询与知识库中的文档统一映射到同一个连续的向量空间中，使语义相近的内容在该空间中彼此靠近；
- **语义相似度计算**：基于向量之间的相似度度量（如余弦相似度），快速检索出与用户查询在语义上最相关的文档片段。

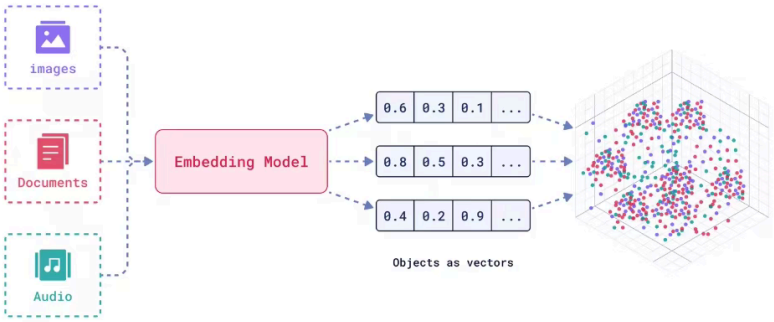


目前主要的 Embedding 类型主要包括：

- **文本 Embedding：**专门用于处理纯文本数据，通过模型（如 Sentence-BERT、DPR、BGE 等）将句子或段落编码为固定维度的句向量。这是当前 RAG 系统中最广泛采用的嵌入形式；



- **多模态Enmedding：**能够联合编码多种模态的信息（如文本、图像、音频等），将不同模态的内容映射到一个共享的语义向量空间，适用于图文检索、跨模态问答等更复杂的应用场景。



- 查询转换（Query Transformation）

查询转换是在用户输入原始问题后、正式发起检索前，对查询语句进行语义澄清、结构优化或意图补全的过程。它的核心作用是弥合用户自然语言表达与知识库专业术语/结构之间的语义鸿沟——通过生成更精准、更完整、更适合检索系统理解的查询形式，显著提升召回结果的相关性与覆盖率。主要方法包括：

1. 查询重写（Query Rewriting）

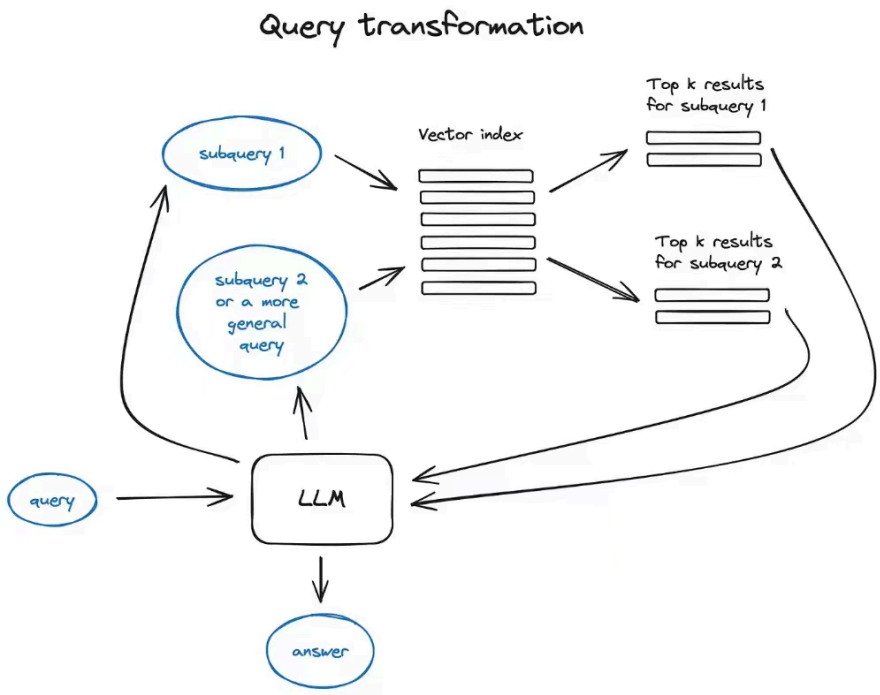
核心思想：将用户模糊或笼统的问题转化为更具体、细节更丰富的表述，明确关键意图（如受众、目标、范围等），从而引导检索系统返回更精准、实用的信息，避免因问题过于宽泛而得到泛泛而谈的结果。

示例：

- 用户原问题：“怎么学Python？”
- 重写后的问题：“零基础初学者如何系统学习Python编程语言？推荐的学习路径和资源有哪些？”

2. 子查询拆解（Sub-query Decomposition）

核心思想：将包含多个意图或维度的复合型问题拆解为若干个单一、明确的子问题，分别进行检索后再综合答案，确保每个关键方面都被充分覆盖，防止因整体匹配失败或注意力分散而导致重要信息遗漏。



示例：

- 用户原问题：“购买新能源汽车时需要考虑哪些因素？补贴政策和充电设施是否完善？”
- 拆解为：
  - “选购新能源汽车时应重点关注哪些参数和配置？”
  - “当前国家和地方对新能源汽车有哪些购车补贴政策？”
  - “家用充电桩安装条件和流程是什么？”

3. 回退提问（Step-back Prompting）

核心思想：当原始问题聚焦于具体现象或应用时，先“后退一步”，提出一个更高层次、更通用的背景性问题，以获取支撑性原理或上下文知识，从而为理解或解答原始问题提供更扎实的基础，避免因过度聚焦细节而忽略关键背景信息。

示例：

- 用户原问题：“为什么我的手机电池掉电特别快？”
- 回退问题：“影响智能手机电池续航的主要因素有哪些？”

4. 检索（Retrieval）

检索是RAG系统中连接用户查询与外部知识库的核心机制，其目标是从海量文档中高效、准确地召回与问题最相关的信息片段，为后续大模型生成提供高质量上下文支撑。不同检索范式在语义理解能力、关键词敏感性、计算效率和适用场景上各有侧重，合理选择或融合多种策略对系统整体性能至关重要。

在实际应用中，单一检索方法往往难以兼顾精确匹配、语义理解与关系推理等多维需求。因此，现代RAG系统普遍摒弃“单打独斗”的检索模式，转而采用混合检索（Hybrid Search）——这已成为当前业界的标准实践。

混合检索的核心思想是：并行调用多种互补的检索路径，再通过智能融合策略整合结果，从而在召回广度与排序精度之间取得更优平衡。目前，主流系统通常同时集成以下四类检索方式：

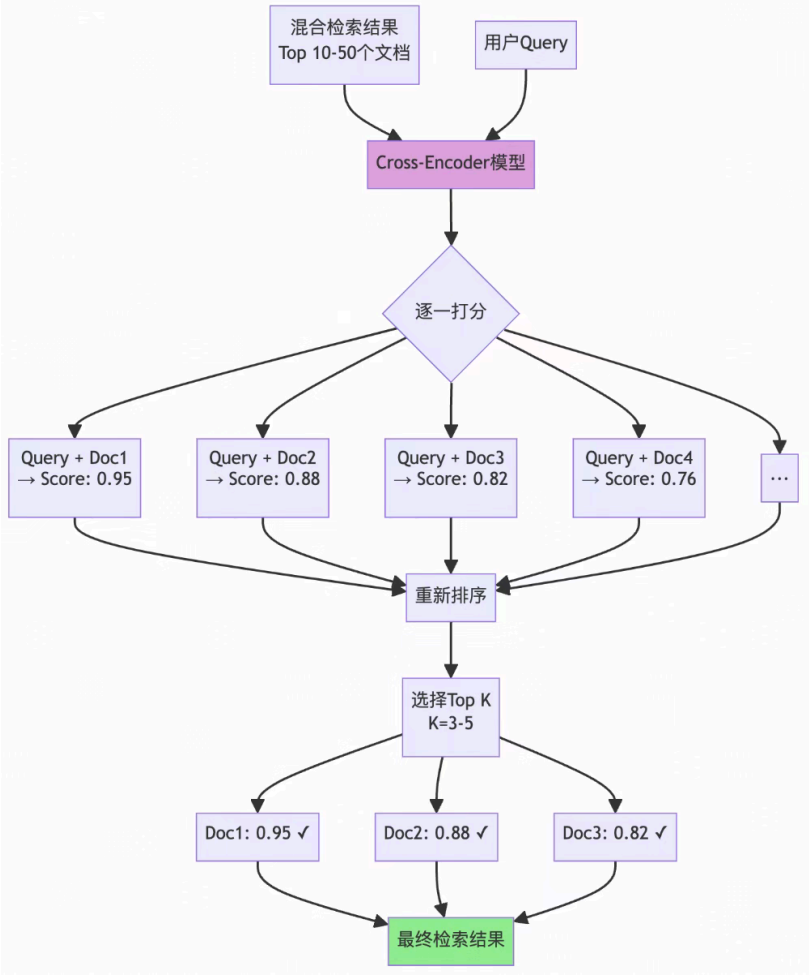
检索方法	原理	优点	缺点	适用场景
全文检索	基于关键词匹配，通过倒排索引快速定位包含查询词的文档，并利用相关性算法（如BM25）对结果进行排序，强调字面匹配和检索效率。	- 查询高效，延迟低 - 工程成熟，可解释性强	- 无语义理解能力 - 对同义词或表达变化不敏感	查询含明确关键词（如人名、术语）
稠密向量检索	使用模型将查询与文档编码为稠密向量，映射到低维、稠密的向量空间中。通过向量相似度匹配语义。	- 强大的语义理解能力	- 计算与存储开销大 - 可解释性差	开放域问答、客服对话等用户表述多样的场景
稀疏向量检索	使用上下文感知的语言模型为每个词项预测其在当前文本中的重要性权重，生成一个高维（词汇表大小，如 30K+）、但绝大多数维度为 0 的稀疏向量。检索仍依赖传统倒排索引，但排序使用学习到的语义权重。	- 兼顾语义与效率	- 语义能力有限 - 依赖训练数据与词表覆盖	兼顾关键词准确性与语义泛化的混合检索需求。
图检索	在结构化的知识图谱上进行检索，将实体作为节点、关系作为边。查询通过图遍历或图神经网络，寻找满足逻辑约束的实体路径	- 支持复杂逻辑推理 - 结构清晰、可追溯	- 构建成本高 - 难以处理非结构化文本	需要多跳推理、强逻辑性的问答任务

- 重排序（ReRanking）

重排序是在初步检索之后，对候选文档按与用户问题的语义相关性重新打分和排序的过程。它的核心作用是提升最终送入大模型的上下文质量——通过更精准地判断“哪些段落真正有用”，过滤掉表面相关但语义不匹配的内容。无论是在多路召回结果融合后，还是单一检索路径下，重排序都能显著改善生成效果。

以目前广泛使用的 Cross-Encoder 重排模型（如 Cohere Rerank、bge-reranker 系列等）为例，其在 RAG 系统中的典型重排流程如下：

- **获取初筛候选：**从前期检索阶段（例如 BM25、稠密向量或稀疏向量召回）的结果中合并并选取 Top-K 个候选文档。
- **构造查询-文档对：**将用户的问题与每个候选文档拼接成一个完整的输入文本，便于模型整体理解两者关系。
- **计算相关性得分：**把每个拼接后的文本对输入 Cross-Encoder 模型。模型会通读整个句子，综合判断问题和文档在语义上是否匹配，并输出一个 0 到 1 之间的相关性分数——比如 0.9 表示高度相关，0.2 表示基本无关。
- **重排序并筛选：**根据得分对所有候选文档从高到低重新排序，最终只保留 Top-N 个最相关的段落（如 N = 5～10），作为上下文送入大语言模型生成答案。



MCP简介：AI 时代的“Type-C 接口”

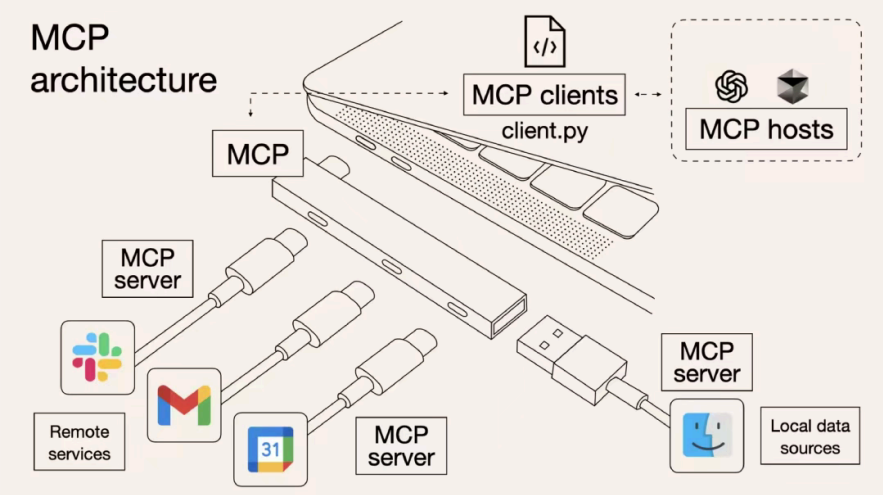
- 什么是MCP?

模型上下文协议（Model Context Protocol，简称MCP） 是一种专为 LLM 和 AI 应用设计的标准化通信协议，旨在解决当前 AI 系统在连接外部工具、数据源和业务逻辑时面临的碎片

化、耦合度高、维护困难等问题。

简言之，MCP 提供了一种通用、动态且自描述的集成机制，使 AI 应用能够安全、灵活地发现并调用各类后端服务的能力，而无需在代码中硬编码具体的接口细节。

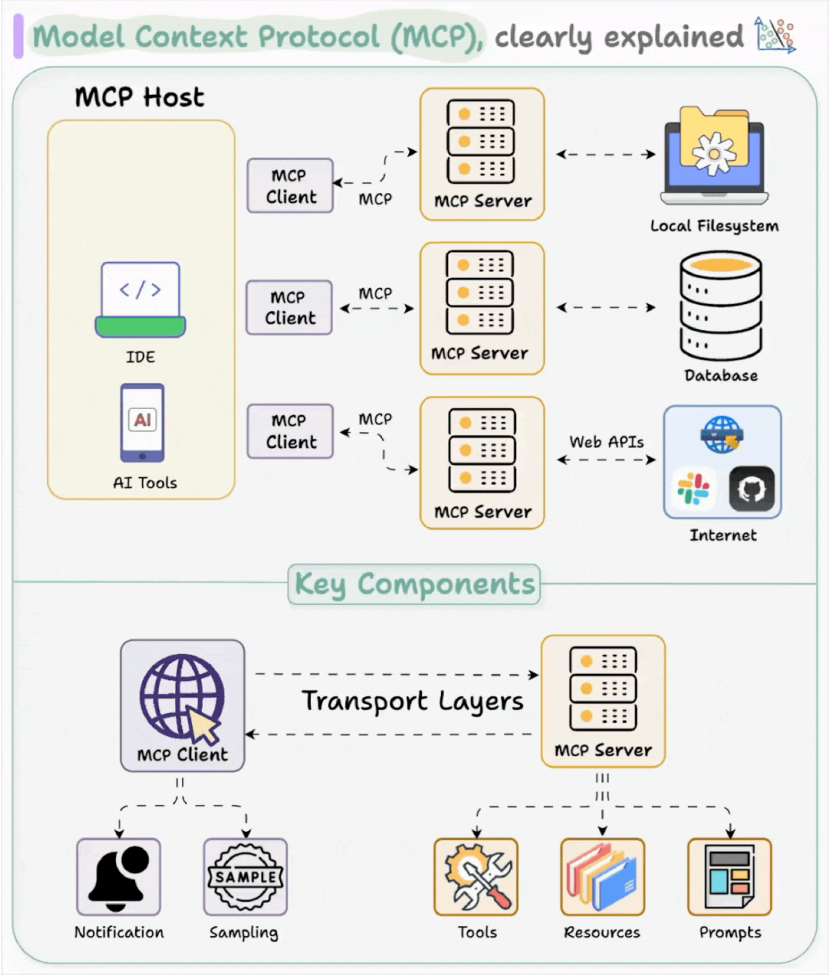
> 类比理解：如果把传统 API 比作“专用充电线”（每台设备需要特定接口），那么 MCP 就是“Type-C”——一个统一、即插即用、支持热插拔和能力协商的智能接口标准。



- 核心架构

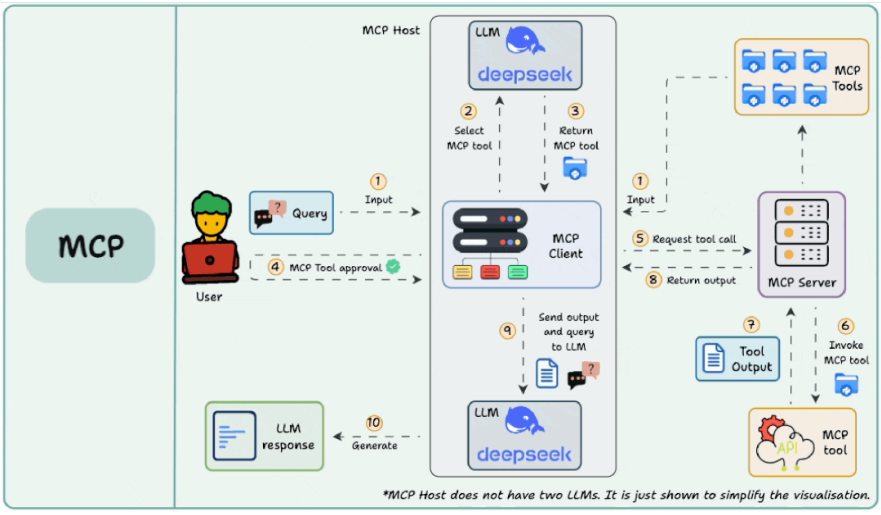
MCP 基于 Client-Server 架构，包含三个角色：

- 主机(Host)：运行 MCP 客户端的 AI 应用。
- 客户端(Client)：在主机内运行，使其能与 MCP 服务器通信。
- 服务器(Server)：暴露特定功能并提供数据访问
  - 工具(Tools)：使 LLM 能够执行具体操作的可调用函数。比如天气查询工具get_weather可以获取指定地点天气
  - 资源(Resources)：向 LLM 公开服务器中的数据和内容。比如知识库文档、数据库记录、配置文件等
  - 提示 (Prompts)：提供可复用的提示词模板。比如翻译提示词模版、数据分析提示词模版等



- 通信流程

MCP 的运行过程可分为两个关键阶段：能力交换与运行时调用。前者实现服务发现与接口的动态协商，后者负责用户请求的安全执行与响应生成，二者协同构建了一个动态、安全且对大语言模型（LLM）友好的工具集成闭环。



阶段一：能力交换

这是 MCP 会话的初始化阶段，通常在 Host 启动或首次连接某个 MCP 服务时触发，用于建立通信并动态发现服务器提供的功能。

- Host 启动 MCP Client 并连接 MCP Server
  - 用户在 AI 应用中启用了某个插件（例如“天气助手”）。
  - Host 启动内置的 MCP Client，并通过预配置的方式（如本地进程、WebSocket URL）连接到对应的 MCP Server（例如一个运行在 `localhost:8080/weather-mcp` 的服务）。
- Client 发送初始化请求
  - MCP Client 向 Server 发送 `initialize` 请求

```
{
  "method": "initialize",
  "params": {
    "clientInfo": { "name": "Claude Desktop", "version": "1.5" },
    "capabilities": { "supportsPrompts": true }
  }
}
```

- Server 返回自身能力清单
  - MCP Server 响应，声明它能提供哪些 Tools（工具）、Resources（资源） 和 Prompts（提示模板），每个都附带结构化元数据（如参数 schema、描述、权限等级）：

```
{
  "result": {
    "serverInfo": { "name": "Weather MCP Server", "version": "2.1" },
    "capabilities": {
      "tools": [
        {
          "name": "get_weather",
          "description": "获取指定城市和日期的天气预报",
          "parameters": {
            "type": "object",
            "properties": {
              "location": { "type": "string", "description": "城市名称"
              "date": { "type": "string", "format": "date", "descripti
            },
            "required": ["location"]
          },
          "requiresUserConsent": false  // 可配置是否需要用户确认
        }
      ],
      "resources": [],
      "prompts": []
    }
```

```
  }
}
```

- Client 注册能力并完成握手
  - MCP Client 解析响应，将 `get_weather` 工具注册到当前会话的“可用工具集”中。
  - 后续 LLM 在生成回复时，即可“知道”自己可以调用这个工具。

阶段二：运行时调用

当用户发起具体查询时，系统进入运行时阶段，执行工具调用并生成最终回答。以用户提问“明天北京天气怎么样？”为例：

- LLM 分析问题并选择工具
  - Host 将用户输入传递给 LLM。
  - LLM 根据上下文和已注册的工具列表，决定调用 `get_weather`，并生成工具调用指令：

```
{
  "tool_name": "get_weather",
  "arguments": {
    "location": "北京",
    "date": "2025-06-15"
  }
}
```

- MCP Client 检查并请求用户授权（可选）
  - MCP Client 查看该工具的元数据：
    - 若 `requiresUserConsent: true` → 弹出确认框：“AI 想查询北京天气，是否允许？”
    - 若为 `false`（如本例），或用户已设置“自动同意”，则跳过确认。
- Client 向 Server 发起工具调用
  - MCP Client 通过协议通道发送 `call` 请求：

```
{
  "method": "call",
  "params": {
    "name": "get_weather",
    "arguments": { "location": "北京", "date": "2025-06-15" }
  }
}
```

- Server 执行工具并返回结果

```
{
  "result": {
```

```
    "temperature": 28,
    "unit": "°C",
    "condition": "Sunny",
    "summary": "北京明天晴，气温 28°C"
  }
}
```

- Client 将结果注入 LLM 上下文
  - MCP Client 收到结果后，将其与原始用户问题一起提交给 LLM
  - 工具返回："北京明天晴，气温 28°C"
- LLM 生成自然语言回答
  - LLM 综合信息，输出流畅回答
  - LLM返回："明天北京天气晴朗，最高气温 28 摄氏度，适合外出！"



总体架构

为提升 AI Coding 在高频、高复杂度业务场景中的采纳率，我们构建了"SPEC（硬规则） + RAG（软上下文） + MCP（标准化接口）"三位一体的知识增强体系。该体系通过分层协同，兼顾准确性、灵活性与可维护性。

- **本地 SPEC 知识库**
  - **优势：**提供强约束、机器可读的契约规范，确保生成代码在接口、数据格式、异常处理等方面严格符合业务要求；支持自动化验证，行为可预期、可审计。
  - **适用场景：**接口定义、代码流程描述、扩展点契约、编码规范、兜底策略等必须遵守的确定性规则。
  - **集成方式：**直接嵌入 IDE 上下文，无需 MCP 检索，作为 LLM 的基础约束条件实时生效。
- **RAG 知识库**
  - **优势：**动态检索非结构化/半结构化私有知识（如业务领域知识、术语、历史经验），提供语境感知的"软上下文"；按需召回，避免上下文过载；支持低成本持续更新。
  - **适用场景：**需求背景解释、最佳实践参考、隐性规则说明、跨域联调上下文等辅助信息。
  - **集成方式：**通过 MCP 协议封装为标准工具服务，按需调用，避免上下文膨胀。

Spec知识库

本地 SPEC 知识库已在猫超导购 C 端工程中全面落地，形成稳定的工作流：

- 所有 Solution 与通用组件均配套 `.spec/` 目录，明确定义输入输出、实验开关、兜底逻辑等关键要素；
- 团队沉淀的通用 Rules（如日志规范 GuideLog、参数校验模板、稳定性兜底策略）已嵌入 Prompt Rules工程，作为系统级约束；
- agent平台在代码生成前自动加载相关 SPEC 文件，将其转化为内部约束条件，确保产出符合业务语义与系统架构。
- 将猫超闪购系统文档体系划分为三大类
  - 功能规范：整体划分为主流程、解决方案、通用组件和描述性规范四类。
  - 需求生命周期管理：围绕具体需求，集中管理从需求拆解到技术方案的全过程文档，确保需求实现可追溯。
  - 项目级规则：统一定义编码规范、目录结构及导购架构等项目级工程标准。

RAG知识库

针对SPEC无法覆盖的非结构化与半结构化知识（如业务领域术语、历史技术方案、跨团队协作上下文等），同时，也为解决因文档激增导致模型上下文长度不堪重负这一瓶颈，我们启动 RAG 知识库建设，目前处于初步落地与验证阶段。

本阶段的核心目标是：通过统一托管与智能检索，将分散在各业务线的静态知识转化为 AI 可按需调用的动态上下文，从而在不增加模型输入负担的前提下，显著提升生成结果的准确性与业务贴合度。

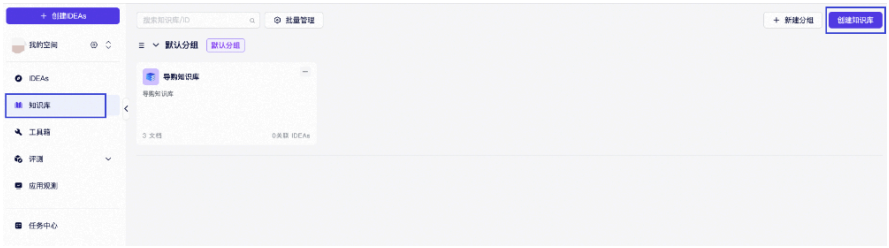
具体进展如下：

- **统一托管与知识整合：**已将部分核心业务文档导入 RAG 知识库；
- **智能检索能力建设：**基于 RAG 提供的向量检索与重排序能力，实现关键词+语义混合召回；该机制能在用户提问时自动理解意图，从海量文档中精准召回最相关的片段；
- **轻量集成与 MCP 封装：**通过 MCP 协议将 RAG 检索能力封装为标准化工具，以“即插即用”的方式嵌入 agent平台工作流；
- **试点场景验证：**在“需求澄清”“错误诊断”“方案参考”等场景中初步验证，能有效补充 SPEC 未覆盖的上下文。

- RAG知识库构建

1. 创建知识库

在RAG平台创建新的知识库实例，作为后续文档管理和检索的基础容器。



2. 知识库配置

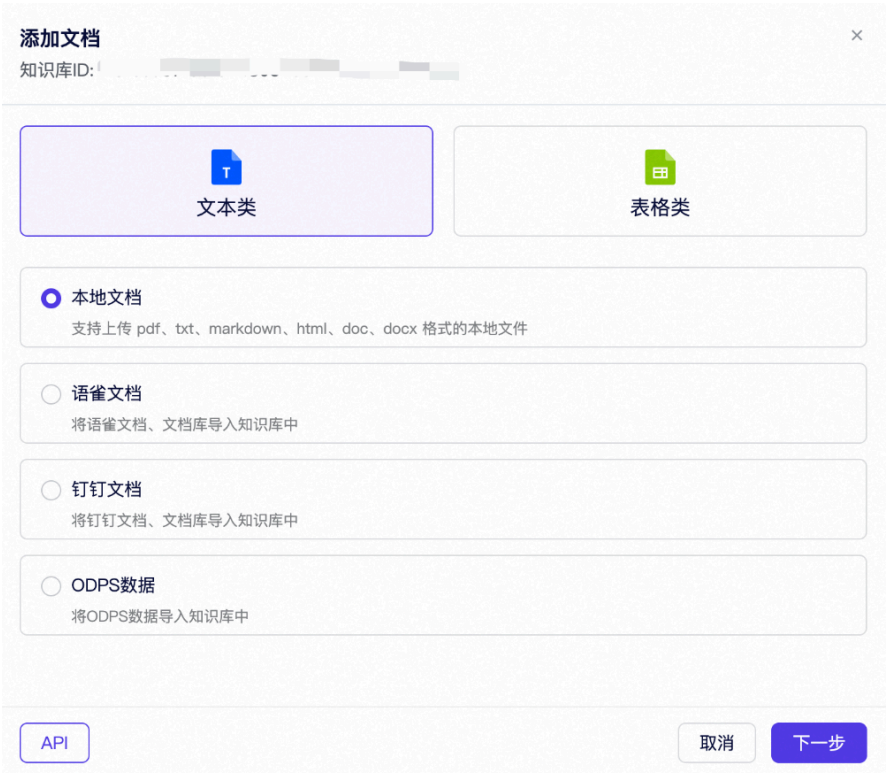
根据实际需求配置知识库参数：

- 向量模型选择：选择适合业务场景的向量化模型
- 文档更新策略：设置文档的自动更新和同步机制



3. 添加文档

支持多种文档格式上传至知识库：

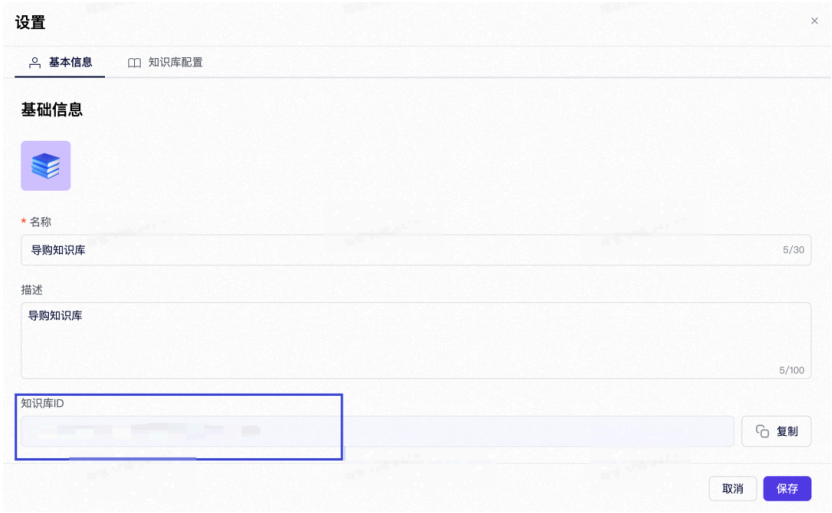




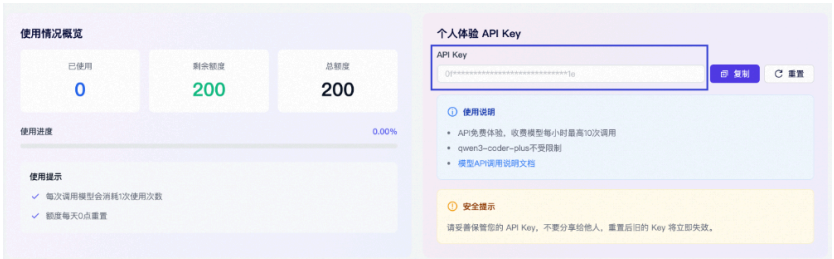
- MCP服务器构建

1. 获取知识库凭证信息

- 获取知识库ID：在知识库详情页面复制知识库唯一标识ID



- 知识库AK绑定
  - 申请知识库AK（免费体验）



  - AK绑定：在知识库设置中完成AK绑定配置



2. 知识库检索API构建

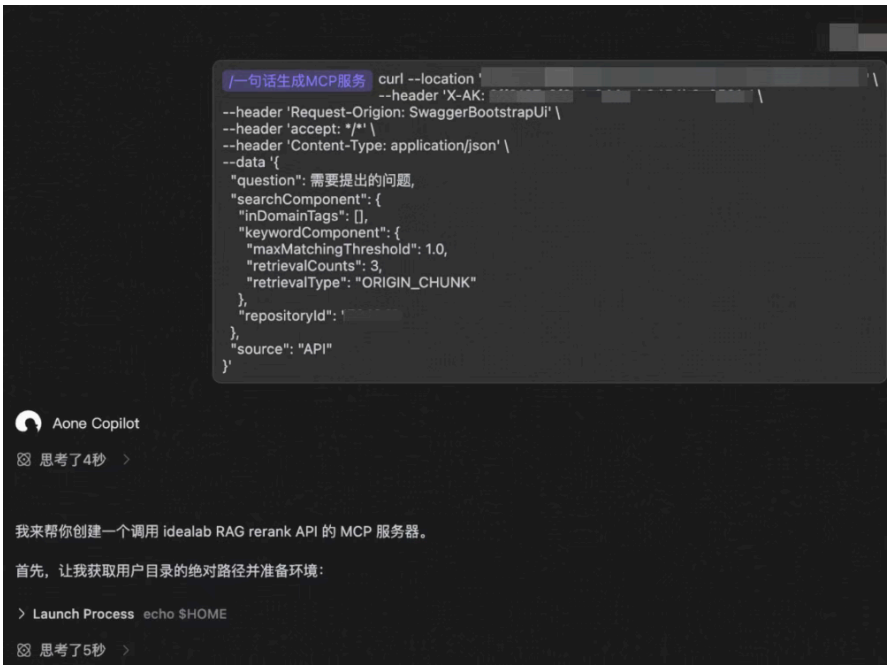
以下是关键词检索的API调用示例（使用上文获取的AK和知识库ID）：

```
curl --location '检索地址' \
--header 'X-AK: AK' \
--header 'Request-Origion: SwaggerBootstrapUi' \
--header 'accept: */*' \
--header 'Content-Type: application/json' \
--data '{
  "question": 需要提出的问题,
  "searchComponent": {
    "inDomainTags": [],
    "keywordComponent": {
      "maxMatchingThreshold": 1.0,
      "retrievalCounts": 3,
      "retrievalType": "ORIGIN_CHUNK"
    },
    "repositoryId": "知识库ID"
  },
  "source": "API"
}'
```

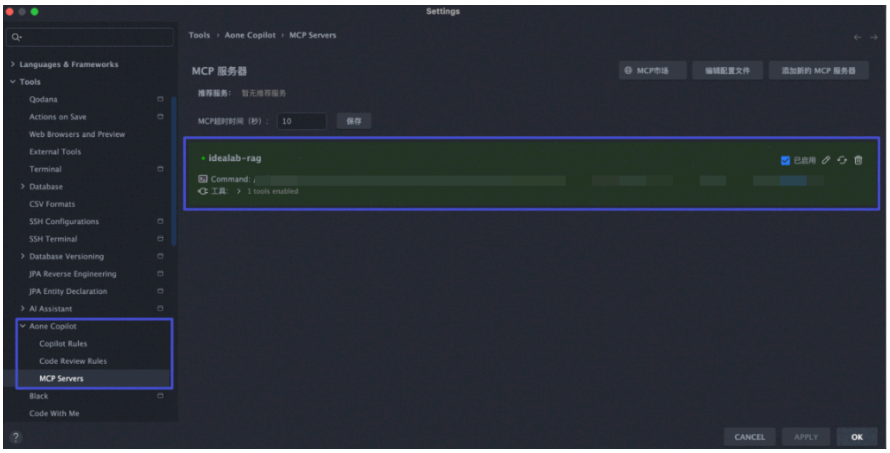
3. MCP服务器快速部署（渐进式策略）

采用“本地试点 → 云端迁移”策略，确保稳定性与可维护性。

- 通过“一句话生成MCP服务”prompt，快速构建本地MCP服务



- agent平台集成验证



- agent平台实战应用

1. 实际提问



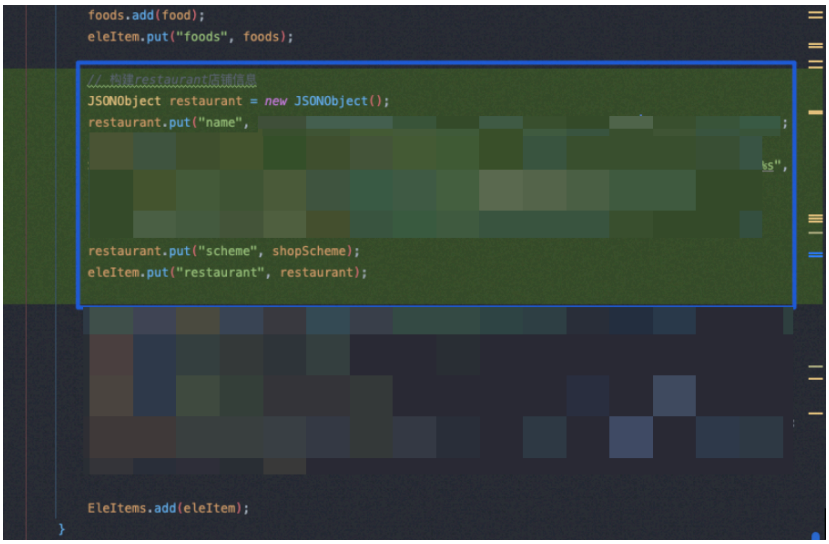
2. 检索质量验证

对比检索过程中返回的RAG知识库片段：



3. 代码生成

基于MCP检索召回的知识库内容，能够准确生成符合业务逻辑和技术规范的代码



04 后续规划

目前，AI Coding 的知识支撑体系已初步落地：SPEC 知识库在规范驱动开发中持续发挥作用，RAG 知识库也初步验证了其在补充非结构化上下文方面的可行性。在此基础上，我们设想下一阶段可围绕以下方向逐步探索优化，尝试构建一个更高质量、更具扩展性的智能知识底座：

- **SPEC 知识库将持续完善：**进一步优化spec目录的分层结构，提升规范的可发现性与复用性；探索建立自动保鲜机制，通过代码变更感知等方式，确保 SPEC 与实现同步演进，避免“规范滞后于代码”。进一步夯实 AI Coding 的“硬规则”基础。
- **RAG 知识库将重点优化以下方向，**以提升动态上下文的质量与覆盖：
  - **向量模型调优：**评估并引入更契合技术文档语义的嵌入模型，提升语义匹配精度与跨文档关联能力；
  - **检索效果增强：**支持关键词与向量混合检索，引入重排序（re-ranking）等机制，提高召回结果的相关性与实用性；
  - **业务知识库持续建设：**在现有基础上，逐步丰富各业务域的核心文档（如导购、推荐、渲染等领域的规范、流程），朝着覆盖全面、结构清晰、易于维护的领域知识体系演进。
  - **MCP 服务云端化：**若本地验证顺利，将把当前基于本地部署的 MCP 检索服务迁移至云端平台。

05 团队介绍

本文作者翔旭，来自淘天集团-导购&商详团队。我们致力于打造智能化、体系化的天猫超市消费者导购链路，通过产品架构设计与持续优化，构建覆盖私域全链路的导购产品体系。我们深度融合AI技术，积极探索内容化、场景化等创新导购形态，为用户提供更精准、更沉浸的购物

体验，同时为猫超自营业务提供从B端到C端的一体化导购解决方案，驱动业务增长与商业价值提升。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 6081

留言

写留言

还没有留言，来写首评